ZG549 API-DRIVEN CLOUD NATIVE SOLUTIONS

API Basics

REST, OpenAPI, and GraphQL — the shared rulebook behind every app you use

BUILT FROM: THE SESSION 1 HANDOUT, SYLLABUS, SLIDE DECK AND RECORDING · EVERY EXAMPLE WORKED OUT IN FULL

How to use this companion. This companion explains APIs from scratch, in plain English. Every idea starts with an everyday picture. It is then built up one step at a time, and every example is worked out in full. Small labels mark the boxes. **THE INTUITION** states the core idea in one breath. **EVERYDAY PICTURE** ties it to real life before any jargon. **WORKED EXAMPLE**, on a grey background, solves a problem with no skipped steps. **WATCH OUT** flags the common mistake. **WAIT — REALLY?** holds a result that sounds wrong but is true. **KEY TAKEAWAY** is the one sentence to remember. Read it in order.

1 Read this request before you read any definition

Every API call has the same three-part shape: a verb, a path, and an answer. Learn to read that shape and the rest of this companion is detail.

Here is a real exchange between a phone app and a server. Do not worry about the words yet. Just read it.

> EVERYDAY PICTURE
>
> **Request:**
> `GET /walkers/101`
> **Response:**
> `200 OK`
> `{ "id": 101, "name": "Priya", "rating": 4.8, "available": true }`

Try to answer three questions before reading on. What is the app asking for? What did the server say back? How do you know the request worked?

The app asked to *get* the walker with ID `101`. The server answered with `200 OK` — the universal "it worked" code. Along with it came a small block of data describing that one walker. You just read an **API call**: one **verb** (`GET`), one **path** (`/walkers/101`), and one **status code** plus **body** coming back. Every REST request you will ever see follows this same shape. The rest of this companion just fills in the vocabulary. Which verbs exist, what the status codes mean, and how two teams agree on a shape up front.

KEY TAKEAWAY

An API call is a verb plus a path going out, and a status code plus a body coming back. Everything else in this companion is detail on top of that one shape.

2 An API is a contract, not a machine

The client never sees how the server works inside. It only sees the promise the API makes. A restaurant's menu works the same way: it promises a dish, never the kitchen.

An **API** (Application Programming Interface) is a structured way for two pieces of software to talk. It fixes four things in advance. The **HTTP method** to use. The **URL** to call. What **authentication** is needed. What **data format** comes back. Nothing else is promised, and nothing else is needed.

THE INTUITION

A menu names a dish and its price. It never shows you the stove, the chef, or the walk-in fridge. An API is the same kind of promise: it names an operation and its shape, and hides everything behind it. The hidden part can change completely — a new database, a rewritten server. The promise still holds either way, so the client never has to change.

EVERYDAY PICTURE

Picture *PawPatrol*, a dog-walking booking app. You open it and ask "is Priya free to walk Buddy on Saturday?" Your phone sends one request. It has no idea how PawPatrol finds the answer: a database, a calendar service, or even another company's API. It does not need to know. It only needs the promised shape of the question and the answer.

WORKED EXAMPLE · *the weather API, traced end to end*

Suppose an app wants today's weather for a city.

Step 1 **Pick the method.** Reading data, so the method is `GET` — never `POST`, which is reserved for creating something new.

Step 2 **Pick the URL.** The API's documentation says `GET /weather?city=Pune`. The city name is a query parameter.

Step 3 **Send the request.** The client never asks how the server gets the number. A live sensor feed, a forecasting model, or another company's API all look the same from outside.

Step 4 **Receive the response.** The server answers `200 OK` with a body such as `{"city": "Pune", "tempC": 27}`.

Step 5 **Use the data.** The app reads `tempC` out of the body and shows "27°C" on

screen. The contract is the only thing the app relied on.

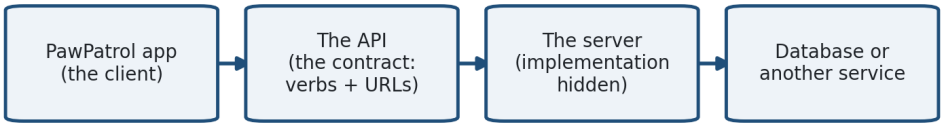


Figure 1. An API is a layer: the client only ever talks to the contract, never to the server's internals.

WATCH OUT

Beginners often think an API is a separate physical box sitting between client and server. It is not. An API is an *interface* — a documented promise — through which the server chooses to expose some of its functionality. There may be no extra hardware at all.

KEY TAKEAWAY

An API fixes the method, the URL, the authentication, and the data format in advance. Everything behind that promise can change freely, and the client never has to care.

Where it's used. Google Maps, Rapid API, and every payment gateway you have used follow this exact pattern. Each puts a documented contract in front of an implementation you never see, and never need to.

3 Synchronous vs. asynchronous: does the caller wait?

Every API style in this companion — REST, GraphQL, gRPC — is synchronous: the caller waits. A message broker is the asynchronous alternative: the caller moves on, and finds out later.

Synchronous means the caller sends a request and does nothing else until the response arrives. **Asynchronous** means the caller hands off the work and continues immediately; the result shows up later, through a separate channel.

THE INTUITION

Picture booking a PawPatrol walker two different ways. Call the support line and stay on hold until someone confirms your slot. That is synchronous: you cannot do anything else while you wait. Now drop Buddy off at daycare in the morning instead. Get a text in the afternoon when he is ready for pickup. That is asynchronous: you went off and lived your day, and got notified later.

EVERYDAY PICTURE

PawPatrol's backend logs every finished walk to an analytics pipeline. The booking service does not wait for analytics to finish before answering the app. It drops one message describing the walk onto a **message broker** — RabbitMQ or Apache Kafka are common choices. Then it moves straight on to the next request. Some other service, whenever it is free, picks that message up and processes it.

WATCH OUT

Asynchronous is not automatically "better." A quick lookup, like checking a walker's rating, is fine done synchronously; the wait is a few milliseconds. A bursty, high-volume job is a different story. Picture a fleet of sensors, each reporting once a second. Forcing every sender to wait would grind the whole system to a halt. Pick based on the shape of the work, not habit.

KEY TAKEAWAY

REST, GraphQL, and gRPC are all synchronous: the caller waits for a direct answer. A message broker is the asynchronous alternative: the caller drops a message and moves on, and something else processes it later.

Where it's used. A fleet of IoT devices reporting sensor readings is the textbook asynchronous case. It is exactly the shape this module returns to in its later sessions on IoT and data analytics.

4 The four questions every REST call answers

*REST (Representational State Transfer) names every piece of data a **resource** and gives it one address. Four verbs then ask what to do with it.*

REST was introduced by Roy Fielding in 2000. In REST, anything the API exposes — a dog walker, a booking, a student, a product — is a **resource**. Every resource has one address, its **URI**. A REST call then answers four questions. *Which resource* (the URI)? *What to do with it* (the verb)? *Did it work* (the status code)? *What does it look like now* (the body, usually JSON or XML)?

THE INTUITION

There are four CRUD operations: Create, Read, Update, Delete. Each maps to one HTTP verb. `POST` creates, `GET` reads, `PUT` or `PATCH` updates, and `DELETE` removes. Learn this mapping and you can read almost any REST API's documentation on sight.

EVERYDAY PICTURE

Back in PawPatrol: a *booking* is a resource at `/bookings/{id}`. Creating a new walk is `POST /bookings`. Checking its status is `GET /bookings/42`. Rescheduling it is `PUT`

`/bookings/42`. Cancelling it is `DELETE /bookings/42`. One resource, one address pattern, four verbs.

WORKED EXAMPLE · *designing PawPatrol's booking API from scratch*

Follow the same seven-stage lifecycle a real team uses, illustrated on the *booking* resource.

Step 1 **Requirements.** A user must be able to create a booking, look one up, change its time, and cancel it. These four needs decide which operations the API must expose — nothing more, nothing less.

Step 2 **Design.** Translate each need into an endpoint: `POST /bookings` (create), `GET /bookings/{id}` (read), `PUT /bookings/{id}` (update), `DELETE /bookings/{id}` (delete).

Step 3 **Configure & publish.** The team picks a framework — say, FastAPI in Python, running on a Uvicorn server. It then publishes the contract, so other teams can start building against it today.

Step 4 **Develop.** Engineers implement each endpoint against the published contract, without waiting for every other part to be finished.

Step 5 **Test.** Every endpoint is checked against the contract: does `POST /bookings` really return `201`? Does a missing booking really return `404`?

Step 6 **Deploy.** The finished API is made available to the real PawPatrol app.

Step 7 **Trace one call.** Take a cancellation. `DELETE /bookings/42` arrives, the server finds booking 42 and removes it. It then answers `200 OK`, or `204 No Content` if there is nothing left to return.

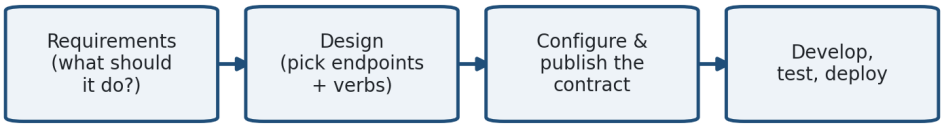


Figure 2. From idea to running API: the seven-stage build lifecycle, shown on the booking resource above.

Status codes, at a glance. The first digit tells the whole story.

Code	Meaning
200 series	success — 200 OK, 201 Created
300 series	redirection
400 series	the client's mistake — 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
500 series	the server's mistake — Internal Server Error

WATCH OUT

`401` and `403` are easy to swap. `401 Unauthorized` means "I don't know who you are" — bad or missing credentials. `403 Forbidden` means "I know who you are, and you're not allowed." The server understood the request both times; only the reason for refusing differs.

WAIT — REALLY?

REST and HTTP are not the same thing, even though they share the same four verbs. HTTP is a communication protocol — the postal service. REST is an architectural style for using it — a set of habits about naming resources and picking verbs. You can send HTTP requests that are not REST at all; REST is simply the disciplined way of doing it.

KEY TAKEAWAY

A REST resource has one address. Four verbs — POST, GET, PUT/PATCH, DELETE — cover create, read, update, and delete. The status code's first digit always tells you who is at fault, if anyone.

Where it's used. Food-delivery apps such as Swiggy and Zomato are classic REST examples. The mobile app is the client, and a REST API sits in front of order, restaurant, and delivery services. The app never has to know how any of them work inside.

5 Why two teams need a shared rulebook: OpenAPI

OpenAPI is not an API. It is a standard way to write down an API's shape so that anyone, and any tool, can read it.

Picture two teams: one builds an API, the other must consume it. Without a written contract, the second team is stuck guessing. Which URLs exist? Which verbs are allowed? What data does each call expect? **OpenAPI** is a specification — a structured, machine-readable document, written in JSON or YAML. It answers all of those questions in one place.

THE INTUITION

OpenAPI is the architectural blueprint, not the building. A human, or a tool such as Swagger, can read that one blueprint. From it, three things get generated automatically: documentation, a client SDK, and tests.

EVERYDAY PICTURE

A tool called Swagger UI can turn an OpenAPI file into an interactive web page. Click a button, send a real request, and see a real response — all without writing a line of code. That page is generated entirely from the blueprint; nobody wrote it by hand.

WAIT — REALLY?

A common first question is "what's the difference between an API and an OpenAPI?" OpenAPI is not itself an API you can call. It is a format for *describing* an API: its endpoints, its verbs, its request and response shapes, and its status codes. The API does the work; OpenAPI writes it down so no one has to guess.

KEY TAKEAWAY

OpenAPI removes the guesswork between two teams. It describes an API's endpoints, verbs, and data shapes in one machine-readable file. Tools then turn that file into docs, test suites, and client code automatically.

6 When REST isn't enough: GraphQL and the N+1 problem

REST hands back a whole resource at a time. GraphQL lets the client ask for exactly the fields it wants, in a single request.

Suppose PawPatrol's app needs a walker's profile, their recent bookings, and the review left on each booking, all on one screen. With REST, that is one request for the profile, plus one for the list of bookings. Then it is one *more* request per booking, just to fetch its review. This is the **N+1 problem**: one profile request, one bookings request, and N more requests for N bookings.

THE INTUITION

GraphQL lets the client send one query. That query names exactly which fields it wants: profile name and rating, each booking's date, each review's text. The server resolves it all in a single round trip, and no field is ever sent that nobody asked for.

EVERYDAY PICTURE

A social-media app showing your profile, your posts, and the comments on each post has the exact same shape. REST turns this into a storm of requests; GraphQL turns it into one.

WORKED EXAMPLE · *counting the requests for a walker with 5 bookings*

Step 1 **REST, request count.** 1 request for the profile, plus 1 for the list of 5 bookings, plus 1 per booking's review. Total: $1 + 1 + 5 = 7$ requests for one screen.

Step 2 **GraphQL, request count.** One query names the profile fields, the bookings, and each booking's review field, all nested in a single document. Total: 1 request, period.

Step 3 **How the server does it.** GraphQL has a pre-agreed **schema** describing every available type and field. The server checks the incoming query against that

schema. A **resolver** function then fetches exactly the requested data for each field, and nothing more.

Step 4 **Writing data.** GraphQL does not reuse `POST`, `PUT`, or `DELETE`. Every write — create, update, or delete — is called a **mutation**; every read is called a **query**.

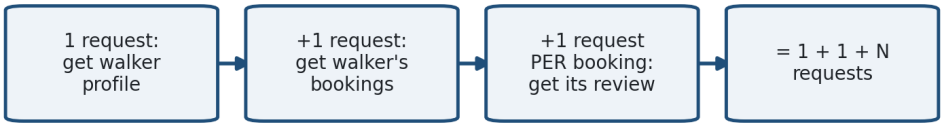


Figure 3. REST's N+1 problem: one profile, one bookings list, and one extra request per booking's review.

WATCH OUT

GraphQL is not automatically asynchronous, and REST is not automatically synchronous. Either style can run either way. In production, GraphQL is usually paired with asynchronous delivery, so the client never blocks while waiting. A **subscription**, for example, pushes a live update the moment a dog walker's location changes. The app never has to ask again and again.

KEY TAKEAWAY

GraphQL trades REST's "fetch the whole resource" habit for "fetch exactly these fields." It collapses an N+1 request storm into one query, at the cost of a schema the server must maintain up front.

Where it's used. AWS offers GraphQL two ways. A fully managed option, AWS AppSync, runs the infrastructure for you. A self-managed option — running your own Apollo or GraphQL server — gives full control instead. Either way, the same single-query idea works on any cloud, not only AWS.

7 gRPC: a strict contract for calls you control

gRPC swaps REST's loose, human-readable JSON for a pre-compiled, strictly typed contract. It is built for speed between services you own, not for a public-facing API.

The Session 1 recording did not have time to reach gRPC or API versioning in depth. The slide deck posted afterward covers both in full, and both are worked through properly below.

gRPC is built for machine-to-machine calls *inside* a system, not for a public, browser-facing API. It uses **Protocol Buffers** (*protobuf*) instead of loose JSON. That is a contract written once, then compiled into fast, strictly typed code for whichever language each side uses.

THE INTUITION

Think of PawPatrol's own backend: the booking service calling the payments service thousands of times a second. Both services are written by the same company and can agree on a rigid, compact format in advance. That is a gRPC-shaped problem. The booking service talking to *your* phone app, written by someone else entirely, is a REST- or GraphQL-shaped problem.

EVERYDAY PICTURE

A **Remote Procedure Call** (RPC) makes a call to another machine look exactly like calling a local function. Your code calls `Add(2, 3)`. Behind the scenes, a **client stub** packages that call into a message and sends it over the network. A **server stub** unpacks it on the other side. The real `Add` function runs, and the result travels back the same way, all invisible to the code that made the call.

THE INTUITION

A `.proto` file is not a program; it is a contract, written once. A line such as `int32 a = 1;` does *not* mean "a equals 1." The number after the `=` is a **field tag**. It fixes which position that field occupies in the binary message. Seat 1 and seat 2 on a bus are fixed the same way, regardless of who is sitting in them. Change a tag later and every client still using the old numbering breaks. That is a breaking change, in the same sense the next section covers.

WORKED EXAMPLE · *one contract, two languages: a calculator service*

Step 1 **Write the contract once,** in a `.proto` file.

```
syntax = "proto3";
package calculator;

service Calculator {
  rpc Add (AddRequest) returns (AddResponse) {}
}

message AddRequest  { int32 a = 1; int32 b = 2; }
message AddResponse { int32 result = 1; }
```

Step 2 **Compile it.** A compiler called `protoc` reads this one file. It generates matching classes and stubs in whichever language each side needs: Python for the server, say, Java for the client.

Step 3 **Call it like a local function.** The Java client calls `calculator.add(2, 3)`. Its generated stub packages the call using the field tags above, and sends it over HTTP/2. The Python server's generated stub unpacks it the same way.

Step 4 **Nobody hand-writes a parser.** Both sides got native classes and methods from the same one contract file. Change the contract, recompile both sides, and they still agree.

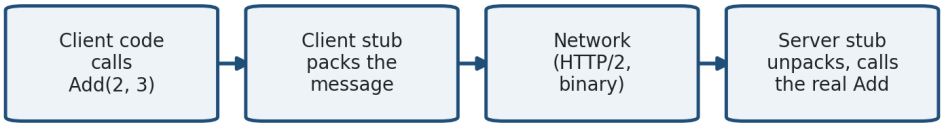


Figure 4. A gRPC call looks local to the caller: a stub on each side hides the network entirely.

> **WATCH OUT**
>
> gRPC's browser support is still limited — most web browsers cannot speak gRPC's binary protocol directly. That is exactly why it is reserved for calls you control on both ends. It is not meant for a public API that users' browsers call straight from JavaScript.

> **KEY TAKEAWAY**
>
> *gRPC trades REST's openness for speed and a strict typed contract. One `.proto` file, compiled once, gives every language a native, type-safe way to call the same service. It is reserved for calls between services you control, not for a public-facing API.*

Where it's used. Fast internal microservice-to-microservice calls, where both ends are written, or at least controlled, by the same team. That is the opposite of the public weather API, or PawPatrol's own customer-facing app, from earlier in this companion.

8 Semantic versioning: safe to upgrade, or not?

A version number alone should tell every caller whether an update is safe to take blindly, or needs checking first.

Semantic versioning numbers a release `MAJOR.MINOR.PATCH` — for example, `v2.1.3`. Each position answers a different question about safety, not just "how new."

> **EVERYDAY PICTURE**
>
> Think of a movie franchise. A sequel that keeps the same characters and story rules is a `MINOR` release. Existing fans keep up fine; new stuff is simply there if they want it. A reboot is different: it changes the core rules, starting a new continuity the old one does not fit into. That is a `MAJOR` release, and old fans, meaning old client code, should expect it to break what they knew. A `PATCH` release just fixes a continuity error without changing the story at all.

A version bump is only worth making `MAJOR` when a change is genuinely breaking. A **breaking change** is one that would make existing client code fail. Breaking changes include:

- changing the response format. JSON becomes XML, say.
- changing a field's data type. A string ID becomes an integer ID.

- renaming or removing a resource or a field.
- adding a new *required* field to the request. Old clients do not know to send it.

Adding a new *optional* field, or fixing a bug without changing the contract, is not breaking. Those are `MINOR` and `PATCH` bumps, and no existing caller is ever forced to change anything.

WORKED EXAMPLE · *versioning a Movies API over time*

Step 1 **v1.0.0.** `GET /api/v1.0.0/movies` ships the initial release: title, director, year, plot summary.

Step 2 **v1.1.0.** `GET /api/v1.1.0/movies/search` adds search by genre or actor. A new, optional capability, so this is a `MINOR` bump.

Step 3 **v1.1.1.** The same endpoint, with a bug in the 1.1.0 search fixed and no new features: a `PATCH` bump.

Step 4 **v2.0.0.** `GET /api/v2.0.0/movies` switches to a new data format and is not backward-compatible. Old clients must update to keep working, so this is a `MAJOR` bump.

Notice that `v1.x` clients keep working unchanged forever; they simply never call the `v2.0.0` endpoint. Only the `MAJOR` bump ever forces anyone to change their code.

WATCH OUT

Bumping `MAJOR` is a promise that old client code *may break*. Bumping `MINOR` or `PATCH` is a promise that it will not. Suppose an API silently changes behaviour on a `PATCH` release. It has broken that promise, no matter how small the change felt.

KEY TAKEAWAY

Semantic versioning tells every caller, from the number alone, whether an update is safe to take blindly. Reserve `MAJOR` for genuine breaking changes: a format change, a type change, a rename, or a new required field. Never reserve it for a change that simply happens to feel big.

Self-test

1. A request arrives with a valid login but the user tries to delete someone else's booking. Which status code fits best?

403 Forbidden — the server knows who you are; you're just not allowed to do this.

2. Why is `POST /weather?city=Pune` the wrong way to ask for today's weather?

Reading data should use GET, not POST. POST is reserved for creating something new; nothing is being created here.

3. A mobile screen needs a walker's name, their 5 most recent bookings, and each booking's review. How many REST calls, and how many GraphQL calls, does that need?

REST: 1 + 1 + 5 = 7 calls. GraphQL: 1 call, naming every field it needs.

4. True or false: OpenAPI is itself a kind of API you can call directly.

False. OpenAPI only describes an API's shape. Documentation tools and code generators read it; it is never called at runtime.

5. A library releases `v3.0.0` after `v2.4.1`. What should every integrator assume, and why?

Assume old code may break. A MAJOR version bump is the project's own warning that it changed a rule old callers relied on.

6. A fleet of 10,000 IoT sensors each report a reading once a second. Should every sensor call a synchronous REST endpoint and wait for a reply, or drop each reading onto a message broker? Why?

A message broker. Forcing 10,000 synchronous waits per second would bottleneck the whole system. A broker lets every sensor move on immediately, and lets consumers work through the backlog at their own pace.

7. In that `AddRequest` message, what does the field tag `= 2` next to `b` mean?

It fixes b's position in the binary-encoded message — a field tag — not b's value. The message carries no value 2 by itself; 2 is only assigned once a real request is sent.

Pocket card

KEY TAKEAWAY

- *An API call is a verb plus a path going out, a status code plus a body coming back.*
- *An API is a contract: method, URL, auth, and data format. The implementation behind it can change freely.*
- *REST: one resource, one URI. Four verbs cover it — POST, GET, PUT or PATCH, DELETE — for create, read, update, delete.*
- *Status codes: 200s succeed, 400s are the client's fault, 500s are the server's fault. 401 means "who are you," 403 means "I know who you are, and no."*
- *OpenAPI describes an API's shape in JSON or YAML so docs, SDKs, and tests can be generated automatically. It is not itself an API.*
- *GraphQL lets a client ask for exactly the fields it needs in one query, avoiding REST's N+1 request storm. Reads are queries, writes are mutations.*

- *Synchronous means the caller waits for a direct answer; REST, GraphQL, and gRPC all work this way. Asynchronous means the caller drops a message on a broker (RabbitMQ, Kafka) and moves on; something else processes it later.*
- *gRPC is a fast, strictly typed protocol for calls between services you control, not for a public-facing API. One `.proto` contract, compiled once, generates native code for every language; field tags (`= 1`, `= 2`) fix wire position, not value.*
- *Semantic versioning (`MAJOR.MINOR.PATCH`) tells callers, from the number alone, whether an update is safe to take blindly. Reserve MAJOR for genuine breaking changes: a format change, a type change, a rename, or a new required field.*

Glossary & symbols

Key terms.

API — a documented contract letting two programs talk without either seeing the other's internals.

resource — anything a REST API exposes — a walker, a booking, a product — identified by one URI.

verb — the action word in a request: GET, POST, PUT, PATCH, DELETE.

status code — a three-digit number telling the caller what happened; its first digit names the category.

OpenAPI — a JSON/YAML specification describing an API's endpoints, verbs, and data shapes, so tools can read it automatically.

GraphQL — a query language letting a client request exactly the fields it needs in a single round trip.

N+1 problem — the request storm created when fetching a list, then fetching one more thing per item in that list, separately.

resolver — the GraphQL server-side function that fetches the data for one requested field.

mutation — a GraphQL write operation — the equivalent of POST, PUT, PATCH, or DELETE in REST.

gRPC — a strict, typed, high-speed protocol for calls between services under one team's control.

semantic versioning — a `MAJOR.MINOR.PATCH` numbering scheme that signals whether a release is safe to take without checking.

synchronous / asynchronous — synchronous means the caller waits for a direct answer. Asynchronous means the caller moves on and finds out the result later, through a separate channel.

message broker a system (RabbitMQ, Apache Kafka, Amazon SQS) that queues messages between services, so the sender never has to wait for the receiver.

RPC Remote Procedure Call — a call to another machine made to look like a call to a local function.

client stub / server stub the generated code on each side of an RPC call that packages and unpacks the message. It hides the network from the code that made the call.

field tag the number after `=` in a `.proto` message field. It fixes the field's position in the binary encoding, not its value.

breaking change any change that would make existing client code fail. Examples: a format change, a type change, a rename, a removal, or a new required field.

Status code families, at a glance.

Range	Who is "at fault," if anyone
200–299	nobody — the request succeeded
300–399	nobody — the client should look elsewhere
400–499	the client — the request itself was the problem
500–599	the server — the request was fine, the server failed